

GIẢI THÍCH MÃ NGUỒN DỰ ÁN

Hệ thống Quản lý Thư viện — Library Management System

Tài liệu mô tả vai trò / chức năng của từng file mã nguồn trong dự án,
tổ chức theo đúng cấu trúc thư mục thực tế của repository.

MỤC LỤC

1. Tổng quan
2. Backend — Khởi động & lỗi hệ thống (app/, app/core/)
3. Backend — Thành phần dùng chung (app/common/)
4. Module Auth — Đăng nhập (app/modules/auth/)
5. Module Users — Tài khoản nhân viên (app/modules/users/)
6. Module Readers — Độc giả (app/modules/readers/)
7. Module Documents — Đầu sách (app/modules/documents/)
8. Module Copies — Bản sao vật lý (app/modules/copies/)
9. Module Borrows — Phiếu mượn/trả (app/modules/borrows/)
10. Module Fines — Tiền phạt (app/modules/fines/)
11. Module Reports — Báo cáo (app/modules/reports/)
12. Backend — Script & file cấu hình khác
13. Frontend — Thành phần lõi (frontend/, frontend/js/)
14. Frontend — Các trang chức năng (frontend/js/pages/)
15. Thư mục còn sót lại: borrows-fines-reports-module/

1. Tổng quan

Dự án gồm 2 phần độc lập: backend (FastAPI + MySQL, cung cấp REST API) và frontend (SPA JavaScript thuần, gọi API qua HTTP). Backend được tổ chức theo module nghiệp vụ — mỗi module nằm trong backend/app/modules// và luôn có tối đa 4 file: model.py (bảng dữ liệu), schema.py (validate request/response), service.py (business logic), router.py (định nghĩa API endpoint).

2. Backend — Khởi động & lỗi hệ thống (app/, app/core/)

File	Vai trò / Chức năng
backend/app/main.py	Điểm khởi chạy FastAPI. Cấu hình CORS, gộp (include_router) toàn bộ 8 module vào app, và trong lifespan sẽ tạo bảng CSDL + seed tài khoản admin mặc định mỗi khi server khởi động.
backend/app/core/config.py	Đọc cấu hình từ file .env (DATABASE_URL, SECRET_KEY, ALGORITHM, thời hạn token) bằng pydantic-settings, cung cấp object settings dùng chung toàn app.
backend/app/core/database.py	Khởi tạo SQLAlchemy engine và SessionLocal; cung cấp dependency get_db() để inject session vào từng request, và init_db() để tạo toàn bộ bảng nếu chưa tồn tại.
backend/app/core/dependencies.py	Các dependency dùng chung cho xác thực/phân quyền: get_current_user() giải mã JWT lấy user hiện tại, require_admin() và require_librarian() chặn truy cập theo vai trò.
backend/app/core/security.py	Hàm băm & so khớp mật khẩu bằng bcrypt (qua Passlib), và tạo/giải mã JWT access token (qua python-jose).

3. Backend — Thành phần dùng chung (app/common/)

File	Vai trò / Chức năng
backend/app/common/constants.py	Hằng số dùng chung: UserRole (admin/librarian) và ErrorMessage (các thông báo lỗi chuẩn hoá, tránh lặp chuỗi rườm rà khắp code).
backend/app/common/responses.py	Chuẩn hoá format phản hồi API: mọi response đều có dạng {success, message, data}, qua 2 hàm success_response() và error_response().

4. Module Auth — Đăng nhập (app/modules/auth/)

File	Vai trò / Chức năng
auth/router.py	Endpoint POST /auth/login — nhận username/password, trả JWT token.
auth/schema.py	LoginRequest (dữ liệu đăng nhập) và TokenResponse (access_token trả về).
auth/service.py	authenticate_user() kiểm tra username/password đúng không; create_token_for_user() sinh JWT chứa user id, username, role.

5. Module Users — Tài khoản nhân viên (app/modules/users/)

File	Vai trò / Chức năng
users/model.py	Model User — bảng users, lưu tài khoản nhân viên (admin/librarian).
users/schema.py	UserCreate (tạo tài khoản) và UserResponse (dữ liệu trả về, không lộ mật khẩu).

File	Vai trò / Chức năng
users/router.py	GET/POST /users — chỉ admin được xem danh sách và tạo tài khoản mới.
users/service.py	create_user() (bấm mật khẩu rồi lưu), get_user_by_username(), và seed_admin() — tạo sẵn tài khoản admin mặc định khi server khởi động lần đầu.

6. Module Readers — Độc giả (app/modules/readers/)

File	Vai trò / Chức năng
readers/model.py	Model Reader — bảng readers: mã độc giả, họ tên, email, phone, trạng thái active/inactive.
readers/schema.py	ReaderCreate, ReaderUpdate, ReaderStatusUpdate (khóa/mở khóa), ReaderResponse, ReaderActiveCheckResponse (module Borrows dùng để kiểm tra độc giả trước khi cho mượn).
readers/router.py	CRUD độc giả + tìm kiếm theo tên/mã + khóa/mở khóa (PATCH /status) + check-active (module khác gọi sang để xác minh độc giả).
readers/service.py	Business logic tương ứng: tạo/sửa độc giả, kiểm tra trùng reader_code/email trước khi tạo mới.

7. Module Documents — Đầu sách (app/modules/documents/)

File	Vai trò / Chức năng
documents/model.py	Model Document — bảng documents: tiêu đề, tác giả, NXB, ISBN, total_copies.
documents/schema.py	DocumentCreate, DocumentUpdate, DocumentResponse.
documents/router.py	CRUD tài liệu, tìm kiếm theo tiêu đề, và 2 endpoint quản lý bản sao của tài liệu (GET/POST /documents/{id}/copies — gọi sang service của module Copies).
documents/service.py	CRUD logic thuần cho bảng documents.

8. Module Copies — Bản sao vật lý (app/modules/copies/)

File	Vai trò / Chức năng
copies/model.py	Model DocumentCopy — bảng document_copies: mỗi dòng là 1 cuốn sách vật lý cụ thể, có copy_code riêng và trạng thái available/borrowed/lost/damaged.
copies/schema.py	CopyCreate, CopyBulkCreate (tạo hàng loạt), CopyStatusUpdate, CopyResponse.
copies/router.py	GET chi tiết 1 bản sao, PATCH đổi trạng thái bản sao.
copies/service.py	create_copy(), create_copies_for_document() (thêm hàng loạt bản sao cho 1 tài liệu và cộng dồn total_copies), update_copy_status().

9. Module Borrows — Phiếu mượn/trả (app/modules/borrows/)

File	Vai trò / Chức năng
borrows/model.py	BorrowOrder (bảng borrow_orders — phiếu mượn) và BorrowItem (bảng borrow_items — từng cuốn sách cụ thể trong phiếu, liên kết tới 1 document_copy).
borrows/schema.py	BorrowCreate, BorrowReturnRequest, BorrowOrderResponse, BorrowItemResponse.

File	Vai trò / Chức năng
borrows/router.py	POST /borrows (tạo phiếu), GET /borrows (danh sách, lọc), GET /borrows/{id} (chi tiết), POST /borrows/{id}/return (trả sách), POST /borrows/{id}/cancel (hủy phiếu).
borrows/service.py	Toàn bộ nghiệp vụ lõi: kiểm tra độc giả/bản sao hợp lệ khi tạo phiếu, tính hạn trả, chuyển trạng thái bản sao, tự sinh khoản phạt khi trả trễ hạn, và tự đóng phiếu (chuyển sang returned) khi mọi bản sao đã được trả.

10. Module Fines — Tiền phạt (app/modules/fines/)

File	Vai trò / Chức năng
fines/model.py	Model Fine — bảng fines: loại phạt (overdue/lost/damaged), số tiền, trạng thái unpaid/paid.
fines/schema.py	FineResponse.
fines/router.py	GET /fines (danh sách, lọc), GET /fines/{id} (chi tiết), PATCH /fines/{id}/pay (đánh dấu đã thanh toán).
fines/service.py	get_all_fines(), get_fine_by_id(), pay_fine() (chặn thanh toán 2 lần cho cùng 1 khoản phạt).

11. Module Reports — Báo cáo (app/modules/reports/)

File	Vai trò / Chức năng
reports/router.py	GET /reports/summary — báo cáo tổng quan toàn hệ thống.
reports/service.py	get_summary_report() — tổng hợp số liệu từ nhiều bảng: tổng độc giả, tổng tài liệu/bản sao, số bản sao đang mượn, tổng phiếu mượn, tổng tiền phạt chưa thu.

12. Backend — Script & file cấu hình khác

File	Vai trò / Chức năng
backend/seed.py	Script độc lập (chạy bằng `python seed.py` sau khi server đã bật) — gọi API để chèn sẵn vài độc giả và đầu sách kèm bản sao mẫu, phục vụ demo nhanh.
backend/requirements.txt	Danh sách thư viện Python cần cài (fastapi, uvicorn, sqlalchemy, pymysql, ...).
backend/.env.example	File mẫu cấu hình môi trường — copy thành .env rồi điền thông tin kết nối MySQL thật.
backend/README.md	Hướng dẫn cài đặt, chạy server, mở Swagger UI, và gợi ý cách chia việc theo module cho 4 người.

13. Frontend — Thành phần lõi (frontend/, frontend/js/)

File	Vai trò / Chức năng
frontend/index.html	Trang HTML gốc duy nhất của SPA — chỉ chứa khung #app rỗng và import toàn bộ file JS theo thứ tự.
frontend/css/style.css	Toàn bộ style giao diện: layout sidebar, bảng dữ liệu, modal, badge trạng thái, toast thông báo, form...

File	Vai trò / Chức năng
frontend/js/api.js	Lớp client gọi REST API bằng fetch — tự động đính JWT token vào header Authorization, tự xử lý khi token hết hạn (401) bằng cách đăng xuất và điều hướng về trang login.
frontend/js/app.js	Router SPA dựa trên hash (#/readers, #/documents...), vẽ layout sidebar theo vai trò user, và các hàm dùng chung: showToast(), openModal()/closeModal(), confirmDialog().
frontend/js/auth.js	Quản lý phiên đăng nhập: lưu/đọc token và user từ localStorage, decode JWT để lấy thông tin user, kiểm tra isLoggedIn()/isAdmin().

14. Frontend — Các trang chức năng (frontend/js/pages/)

File	Vai trò / Chức năng
pages/login.js	Trang đăng nhập — form username/password, gọi auth.login().
pages/dashboard.js	Trang tổng quan — hiển thị các thẻ số liệu thống kê lấy từ GET /reports/summary.
pages/readers.js	Trang quản lý độc giả — bảng danh sách + tìm kiếm, modal thêm/sửa, nút khoá/mở khoá.
pages/documents.js	Trang quản lý tài liệu — CRUD đầu sách + modal riêng để xem/thêm bản sao vật lý cho từng tài liệu (gọi API của module Copies).
pages/borrows.js	Trang mượn/trả — tạo phiếu mượn, xem chi tiết phiếu, trả sách, huỷ phiếu, lọc theo trạng thái.
pages/fines.js	Trang tiền phạt — danh sách khoản phạt, lọc theo trạng thái, nút đánh dấu đã thanh toán.
pages/users.js	Trang quản lý tài khoản nhân viên — chỉ hiển thị trong menu khi đăng nhập bằng tài khoản admin.

15. Thư mục còn sót lại: borrows-fines-reports-module/

File	Vai trò / Chức năng
borrows-fines-reports-module/borrows/*.py borrows-fines-reports-module/fines/*.py borrows-fines-reports-module/reports/*.py	Đây là bản upload gốc (Pull Request #3 — "Add files via upload") của thành viên phụ trách 3 module Borrows/Fines/Reports, nộp dưới dạng thư mục rời rạc ở gốc repo thay vì đúng cấu trúc backend/app/modules/. KHÔNG được backend/app/main.py import hay sử dụng khi chạy server — toàn bộ logic thật đã được tích hợp vào backend/app/modules/{borrows,fines,reports}/ ở các mục phía trên. Thư mục này chỉ còn giá trị tham khảo lịch sử, khuyến nghị xoá để dọn dẹp repo sau khi đã xác nhận không còn thiếu logic nào.